

BÁO CÁO QUÁ TRÌNH THỰC HIỆN LAB_01 NHÓM 4

1. TỔNG QUAN

Mục tiêu xây dựng một dịch vụ lưu trữ dữ liệu trên Kubernetes, đồng thời kiểm soát tài nguyên, phân quyền truy cập, giới hạn network, đo lường hiệu năng và kiểm tra khả năng phục hồi của hệ thống.

Giải pháp được triển khai sử dụng **Kubernetes** kết hợp với **SeaweedFS S3** làm dịch vụ object storage. Dữ liệu được lưu trên PersistentVolumeClaim (PVC) có dung lượng 4 GiB. Dịch vụ S3 được cung cấp thông qua một Service kiểu ClusterIP để giới hạn phạm vi truy cập trong Kubernetes cluster.

Quá trình thực hiện được chia thành 5 nhiệm vụ chính:

1. Thiết lập giới hạn tài nguyên cho namespace.
2. Xây dựng và triển khai storage service.
3. Kiểm tra và chứng minh các giới hạn truy cập.
4. Đo lường hiệu năng truyền dữ liệu dưới các mức concurrency khác nhau.
5. Thực hiện controlled Pod replacement và kiểm tra khả năng phục hồi dữ liệu.

2. PHƯƠNG PHÁP THỰC HIỆN

- Nhóm sử dụng phương pháp triển khai và kiểm thử theo từng giai đoạn. Sau mỗi giai đoạn, nhóm kiểm tra trạng thái bằng giao diện, lưu kết quả dưới dạng file để có thể tái kiểm tra và chứng minh kết quả.
- Quy trình:
Thiết lập guardrails → Triển khai storage → Thiết lập access control → Kiểm thử access boundary → Benchmark → Controlled failure/ Recovery → Thu thập evidence.
- Các công cụ sử dụng:
 - Kubernetes/ Kubectl
 - SeaweedFS S3
 - YAML manifests
 - Python
 - Boto3
 - JSON/ JSONL
 - Git/ Github
- Chương trình [s3lab.py](#) dùng để thực hiện các thao tác S3 như PUT, GET, LIST, DELETE, benchmark.

3. CÁC BƯỚC THỰC HIỆN

3.1. Task 1 - Thiết lập giới hạn tài nguyên.

- Trước khi triển khai storage service, nhóm thiết lập ResourceQuota cho namespace “group04”.
- Quota được cấu hình như sau:

Tài Nguyên	Giới Hạn
CPU requests	2 CPU
CPU limits	4 CPU
Memory requests	2 GiB
Memory limits	4 GiB
Storage requests	8 GiB
PersistentVolumeClaim	2
Pod	10

- Mục đích: Đảm bảo tổng tài nguyên mà các workload trong namespace có thể khai báo không vượt qua phần được cấp.
- Sau khi tạo quota, nhóm thực hiện 2 trường hợp kiểm thử:
 - **Positive case:** Tạo pod với CPU request 100m, memory request 128Mi
—> Hợp lệ
 - **Negative case:** Tạo Pod với CPU request 3, trong khi quota requests.cpu chỉ cho phép tối đa 2
—> Kubernetes từ chối request.
- Điều này chứng minh ResourceQuota thực sự được áp dụng thay vì chỉ tồn tại dưới dạng cấu hình.

3.2. Task 2 - Xây dựng Storage Service.

Nhóm triển khai SeaweedFS dưới dạng Kubernetes Deployment.

- **Persistent Storage**
 - Một PVC object-data được tạo với
 - Dung lượng: 4 GiB
 - Access mode: ReadWriteOnce
 - StorageClass: Standard
 - PVC sau đó được bind thành công và được mount vào storage Pod tại /data.

- Storage Service

- Storage service được triển khai với:
 - Image SeaweedFS được truyền thông qua biến STORAGE_IMAGE.
 - Replicas: 1
 - Strategy: Recreate.
 - Chạy SeaweedFS Mini.
 - Port S3: 8333.
 - Persistent data directory: /data.
 - Authentication configuration được mount từ Secret s3-config.

- Resource request/ limit của storage Pod:

Resource	Request	Limit
CPU	500m	1
Memory	512Mi	1 GiB

- Storage Pod được cấu hình chạy non-root với UID/ GID 1000 và tắt privilege escalation, đồng thời drop toàn bộ Linux capabilities.

- Service

- SeaweedFS được expose thông qua Service:
 - Name: objects
 - Type: ClusterIP
 - Port: 8333
- Do sử dụng ClusterIP, storage service không được expose trực tiếp ra bên ngoài cluster. Sau khi triển khai, nhóm kiểm tra:
 - Pod storage ở trạng thái “Running”
 - Readiness probe hoạt động
 - PVC ở trạng thái “Bound”
 - SeaweedFS khởi động thành công
 - S3 endpoint có thể được truy cập từ các Pod được phép

3.3. Task 3 - Kiểm tra access boundary.

Sau khi storage service hoạt động, nhóm xây dựng các identity và kiểm tra quyền truy cập. Hai bucket được sử dụng:

- Research-raw
- Research-release

Nhóm sử dụng các principal khác nhau để mô phỏng các vai trò: **Owner, Ingstor,**
Analyst, Blocked, Anonymous user.

- **Kiểm thử S3:** Nhóm thực hiện 12 bài kiểm tra từ S01 đến S12.
 - Các kết quả chính:
 - Ingestor: Có thể ghi, đọc dữ liệu trong research-raw.
 - Ingestor: Không được phép ghi/ đọc trong research-release.
 - Analyst: Có thể đọc dữ liệu trong research-release.
 - Analyst: Không được phép ghi hoặc xóa dữ liệu.
 - Analyst: Không được phép truy cập protected bucket.
 - Các request bị từ chối trả về HTTP 403 AccessDenied.
 - Nhóm đã chứng minh được rằng quyền truy cập được giới hạn theo bucket và principal thay vì tất cả client đều có quyền như nhau.
- **Kiểm thử Kubernetes RBAC:** Nhóm tạo ServiceAccount observer với quyền giới hạn.
 - Observer được phép:
 - List Pod
 - Get/ List/ Watch events.
 - Đọc log của Pod.
 - Observer không được phép:
 - Đọc Secret s3-config
 - Tạo Pod
 - Xóa storage Pod.
 - Kết quả K01-K06 đều đạt yêu cầu:
 - K01, K02, K03: Allow.
 - K04, K05, K06: Deny.
 → Điều này chứng minh nguyên tắc least privilege được áp dụng cho Kubernetes RBAC.
- **Kiểm thử NetworkPolicy:** NetworkPolicy private-object-store được cấu hình để chỉ cho phép traffic tới storage Pod từ các Pod có label: “Access: s3” và chỉ trên TCP port 8333.
 - Nhóm thực hiện kiểm tra:
 - Client hợp lệ có thể kết nối tới storage service.
 - Client không hợp lệ thuộc nhóm được phép bị timeout.
 - Kết nối tới port không được mở cũng bị từ chối.
 - DNS vẫn hoạt động bình thường.
 - Các kết quả N01-N04 được lưu dưới dạng file JSON làm bằng chứng.

3.4. Task 4 - Đo lường hiệu năng.

- Nhóm thực hiện benchmark để đánh giá ảnh hưởng của concurrency tới đường truyền dữ liệu.

- Mỗi trial sử dụng:
 - 32 object.
 - Mỗi object: 4MiB.
 - Tổng dữ liệu/ Trial: 128 MiB.
 - Hai mức concurrency: C=1, C=4.
 - Mỗi mức concurrency được chạy 3 lần.
- Tổng cộng 6 Trial:
 - C1: r1-c1, r2-c1, r3-c1
 - C4: r1-c4, r2-c4, r3-c4

Concurrency	Median Put goodput	Median Get goodput
C = 1	92.741 MiB/s	292.983 MiB/s
C = 4	99.810 MiB/s	213.353 MiB/s

- Tính đúng đắn của dữ liệu:
 - PUT thành công: 192/ 192
 - GET thành công: 192/ 192
 - Request thất bại: 0
 - Hash mismatch: 0

3.5. Task 5 - Controlled Pod Replacement và Recovery.

- Để kiểm tra khả năng phục hồi, nhóm thực hiện controlled replacement đối với storage Pod. Trước khi replacement, nhóm ghi lại:
 - Thông tin storage Pod
 - PVC
 - Dữ liệu đang tồn tại
 - Kết quả canary request
- Sau đó storage Pod được xóa. Deployment tự động tạo Pod mới. Nhóm tiếp tục theo dõi trạng thái Pod và thực hiện canary GET request để xác định thời điểm dịch vụ hoạt động trở lại.
- Theo file canary.jsonl:
 - Thời điểm bắt đầu lỗi: 37.13s
 - Thời điểm dịch vụ ổn định trở lại: 44.16s
 - Số dòng lỗi EndpointConnectionError: 7
 - Thời gian gián đoạn: 44.16 - 37.13 = 7.03s.

→ Giá trị này nhỏ hơn giới hạn 120s yêu cầu.
- Sau recovery, nhóm kiểm tra lại dữ liệu và PVC. Dữ liệu vẫn tồn tại nhờ persistent storage, chứng minh việc thay thế Pod không làm mất dữ liệu lưu trên PVC.

4. KẾT QUẢ CUỐI CÙNG.

- Qua 5 task, nhóm đã xây dựng được 1 storage service theo yêu cầu bài thực hành.
- Toàn bộ kết quả được lưu trong evidence trong repository nhằm đảm bảo quá trình thực hiện có thể kiểm tra và tái lập.

5. PHÂN CÔNG VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN.

Thành viên	Phần phụ trách	Công việc	Đóng góp
Đỗ Đức Kiệt	Task 1	Tạo quota, positive.yaml và negative.yaml thực hiện test Pod được chấp nhận và Pod bị từ chối do vượt quota, lưu evidence	20%
Dương Hải Khôi	Task 2	Tạo secret, PVC, Deployment, Service; seed 4 file mẫu, hỗ trợ kiểm tra 22 test trong task 3	20%
Mai Quang Diệp	Task 3	Tạo file test s3lab.py , kiểm tra các bài test bảo mật	20%
Phạm Chí Dũng	Task 4	Xây dựng benchmark, thực hiện 6 trials, tính goodput/p95 và phân tích kết quả	20%
Đinh Huy Hoàng	Task 5	Thay thế Pod lưu trữ bằng lệnh kubectl delete pod, kiểm tra toàn vẹn dữ liệu sau khi khởi động lại pod mới, tính thời gian gián đoạn T_observed.	20%